chapter 6 Memory Management

1. Introduction to Memory Management

- Memory management is an important function of an operating system (OS) responsible for managing the computer's main memory (RAM). It controls how memory is allocated to processes, how it is used, and how it is released when programs finish executing.
- Memory management ensures that multiple programs can run efficiently while sharing the available memory.

Objectives of Memory Management

The main objectives include:

1. **Memory allocation:** Assigning memory space to programs and processes.

2. **Memory deallocation**: Releasing memory when processes finish execution.

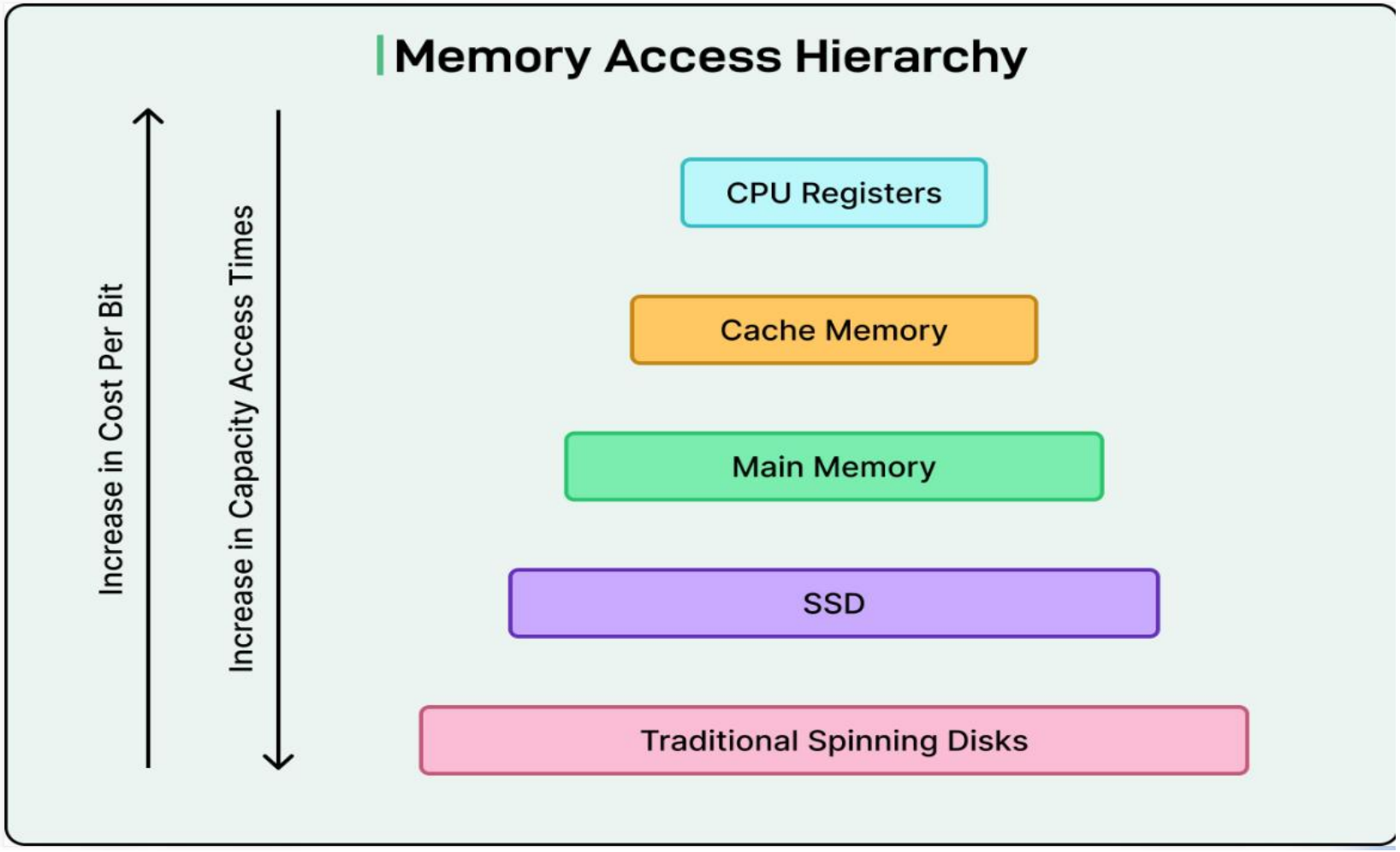
3.**Memory protection:** Preventing one process from accessing another process's memory without permission.

4. **Memory sharing:** Allowing processes to share memory when necessary.

5. **Efficient memory utilization**: Minimizing memory wastage.

6. **Logical and physical memory management:** Managing the relationship between program addresses and actual RAM locations.

2. Types of Computer Memory



2.1 Primary Memory

Primary memory is directly accessible by the CPU.

Examples:

- RAM (Random Access Memory)
- ROM (Read Only Memory)
- Cache memory
- CPU registers

2.2 Secondary Memory

Secondary memory stores data and programs permanently.

Examples:

- Hard Disk Drive (HDD)
- Solid State Drive (SSD)
- USB flash drives
- Memory cards

Difference between primary and secondary memory

Primary Memory	Secondary Memory
Directly accessed by CPU	Accessed through I/O operations
Generally faster	Generally slower
Limited capacity	Usually larger capacity
RAM is volatile	Usually non-volatile
Stores active programs and data	Stores files and applications

3. Memory Allocation Techniques

Memory allocation refers to assigning available memory space to processes.

There are two major approaches:

Contiguous Memory Allocation

Non-Contiguous Memory Allocation

3.1 Contiguous Memory Allocation

In contiguous memory allocation, each process is assigned one continuous block of memory.

There are three common techniques:

a) Fixed Partitioning

- Memory is divided into fixed-size partitions.

- Each partition can hold one process.

- The partition size does not change.

It may cause internal fragmentation.

3.1 Contiguous Memory Allocation

b) Variable Partitioning

- Memory is divided into partitions according to process requirements.
- Processes receive different memory sizes.
- It reduces internal fragmentation but may cause external fragmentation.

c) Dynamic Partitioning

- Memory partitions are created dynamically when processes are loaded.
- Memory is allocated according to process requirements.

3.2 Non-Contiguous Memory Allocation

In non-contiguous allocation, a process can be stored in different locations in physical memory.

Examples:

- **Paging**
- **Segmentation**
- **Segmentation with paging**

4. Memory Allocation Strategies

The operating system uses different strategies to allocate memory.

1. **First Fit:** Allocates the first available memory block large enough for the process.

2. **Best Fit**: Allocates the smallest available block that can accommodate the process.

3. **Worst Fit:** Allocates the largest available memory block.

4. **Next Fit:** Similar to First Fit, but searches from the location of the previous allocation.

Example

Suppose available memory blocks are:

Block	Memory size
Block 1	100 KB
Block 2	500 KB
Block 3	200 KB
Block 4	300 KB
Block 5	600 KB

A process requires 212 KB.

- - First Fit selects Block 2 (500 KB).
- - Best Fit selects Block 4 (300 KB).
- - Worst Fit selects Block 5 (600 KB).

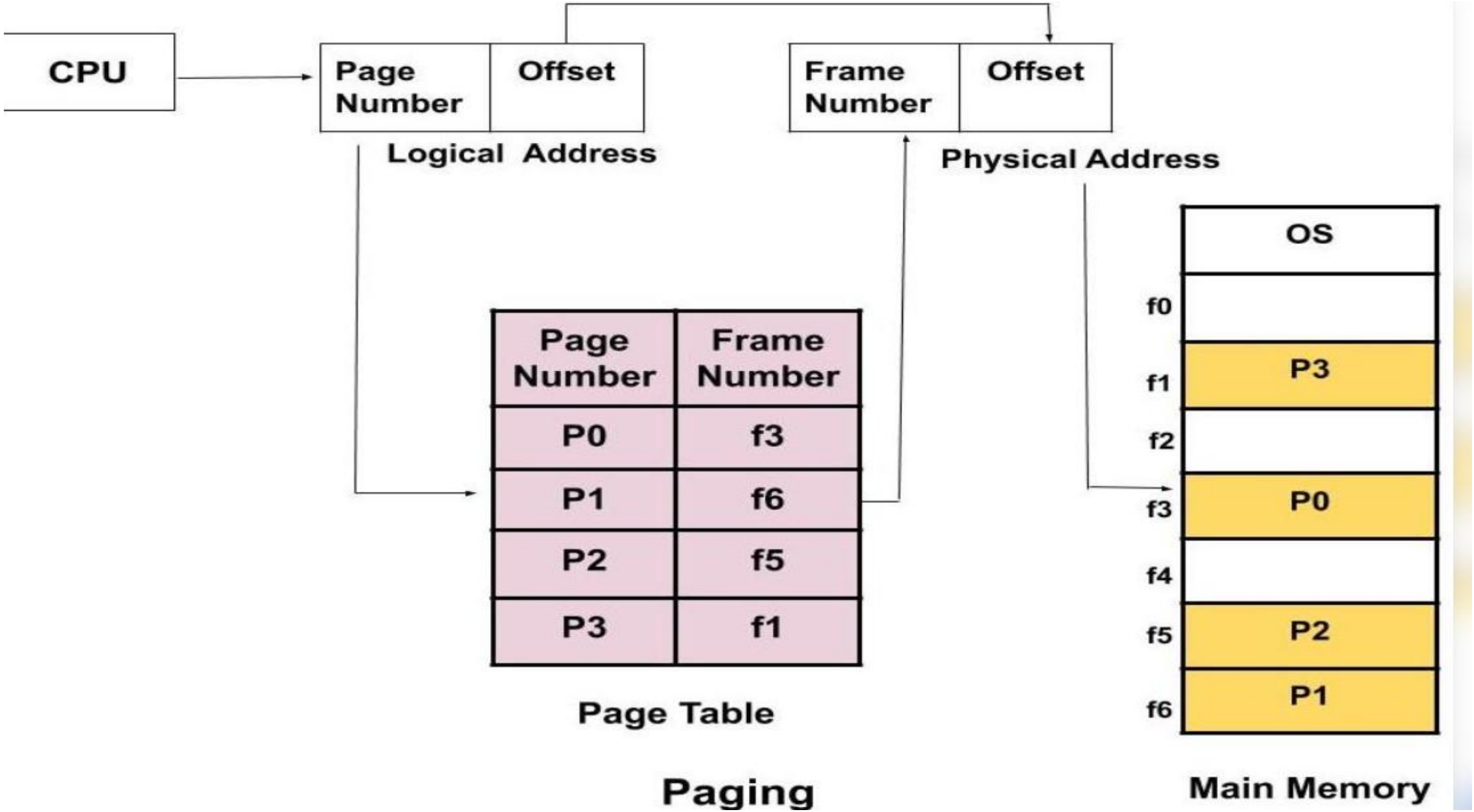
5. Paging

Paging is a memory management technique that divides:

- **Logical** memory into fixed-size blocks called **pages.**
- **Physical** memory into fixed-size blocks called **frames.**

Pages are loaded into available frames in physical memory.

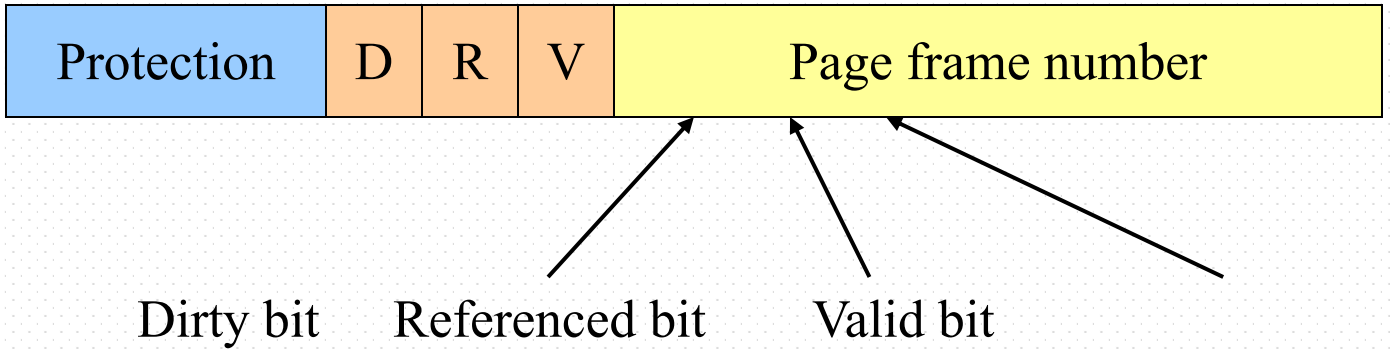
A page table maintains the relationship between pages and frames.


CPU
Page Number
Offset
Logical Address
Frame Number
Offset
Physical Address
Page Number
Frame Number
P0
f3
P1
f6
P2
f5
P3
f1
Page Table
OS
f0
f1
P3
f2
f3
P0
f4
f5
P2
f6
P1
Main Memory

Paging

What's in a page table entry?

- Each entry in the page table contains
  - Valid bit: set if this logical page number has a corresponding physical frame in memory
    - If not valid, remainder of PTE is irrelevant
  - Page frame number: page in physical memory
  - Referenced bit: set if data on the page has been accessed
  - Dirty (modified) bit :set if data on the page has been modified
  - Protection information

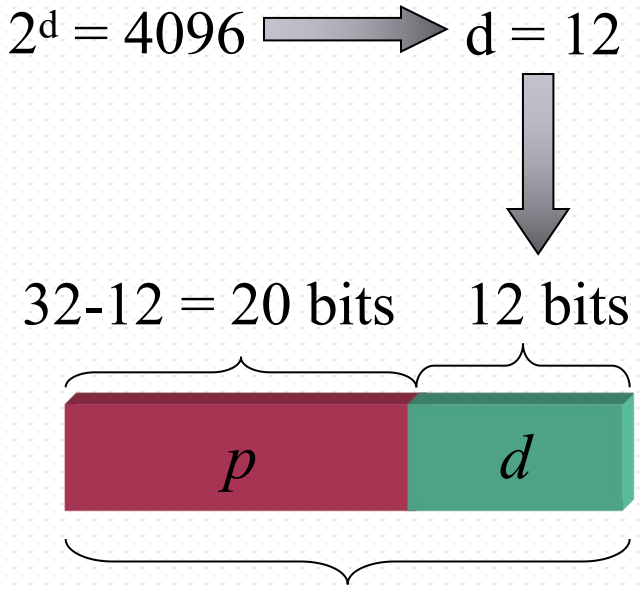


Mapping logical => physical address

- Split address from CPU into two pieces
  - Page number (*p*)
  - Page offset (*d*)
- Page number
  - Index into page table
  - Page table contains base address of page in physical memory
- Page offset
  - Added to base address to get actual physical memory address
- Page size = 2^d bytes

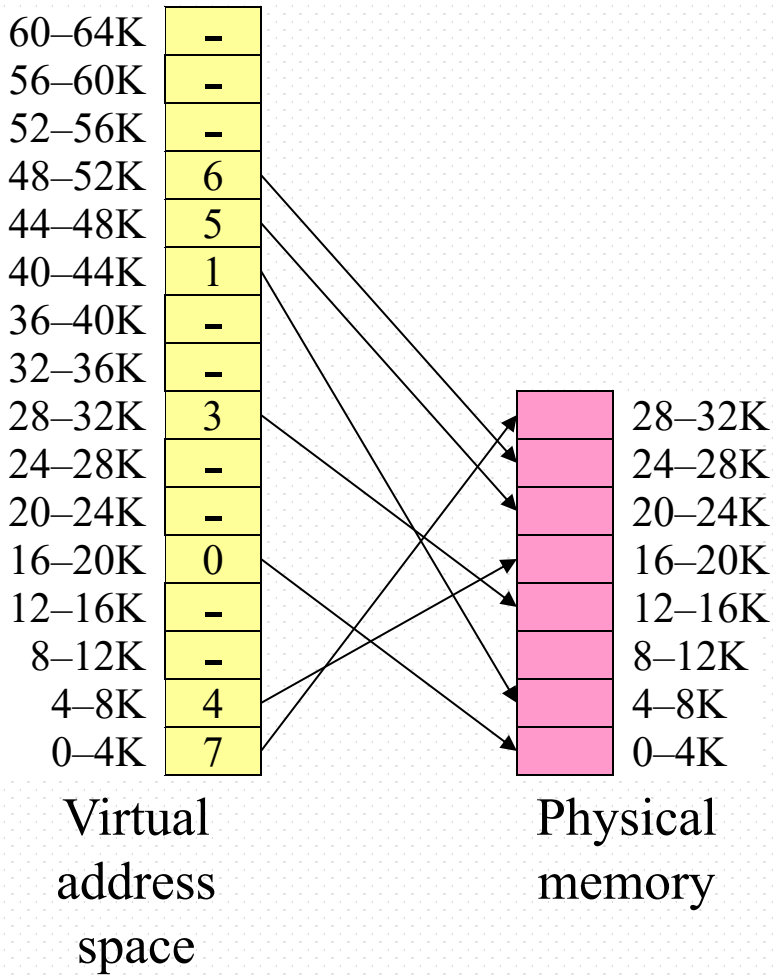
Example:

- 4 KB (=4096 byte) pages
- 32 bit logical addresses



Paging and page tables

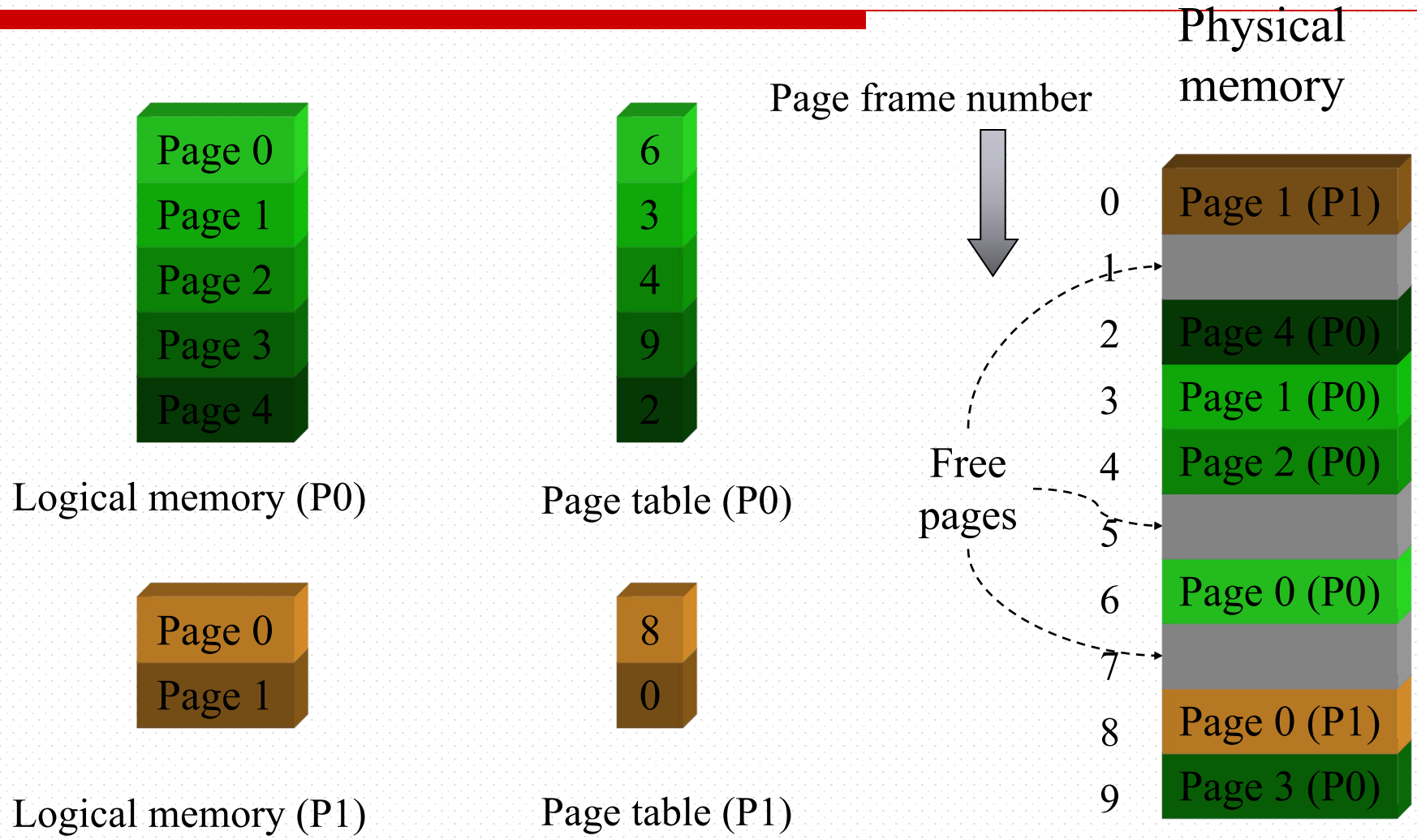
- Virtual addresses mapped to physical addresses
  - Unit of mapping is called a *page*
  - All addresses in the same virtual page are in the same physical page
  - *Page table entry* (PTE) contains translation for a single page
- Table translates virtual page number to physical page number
  - Not all virtual memory has a physical page
  - Not every physical page need be used



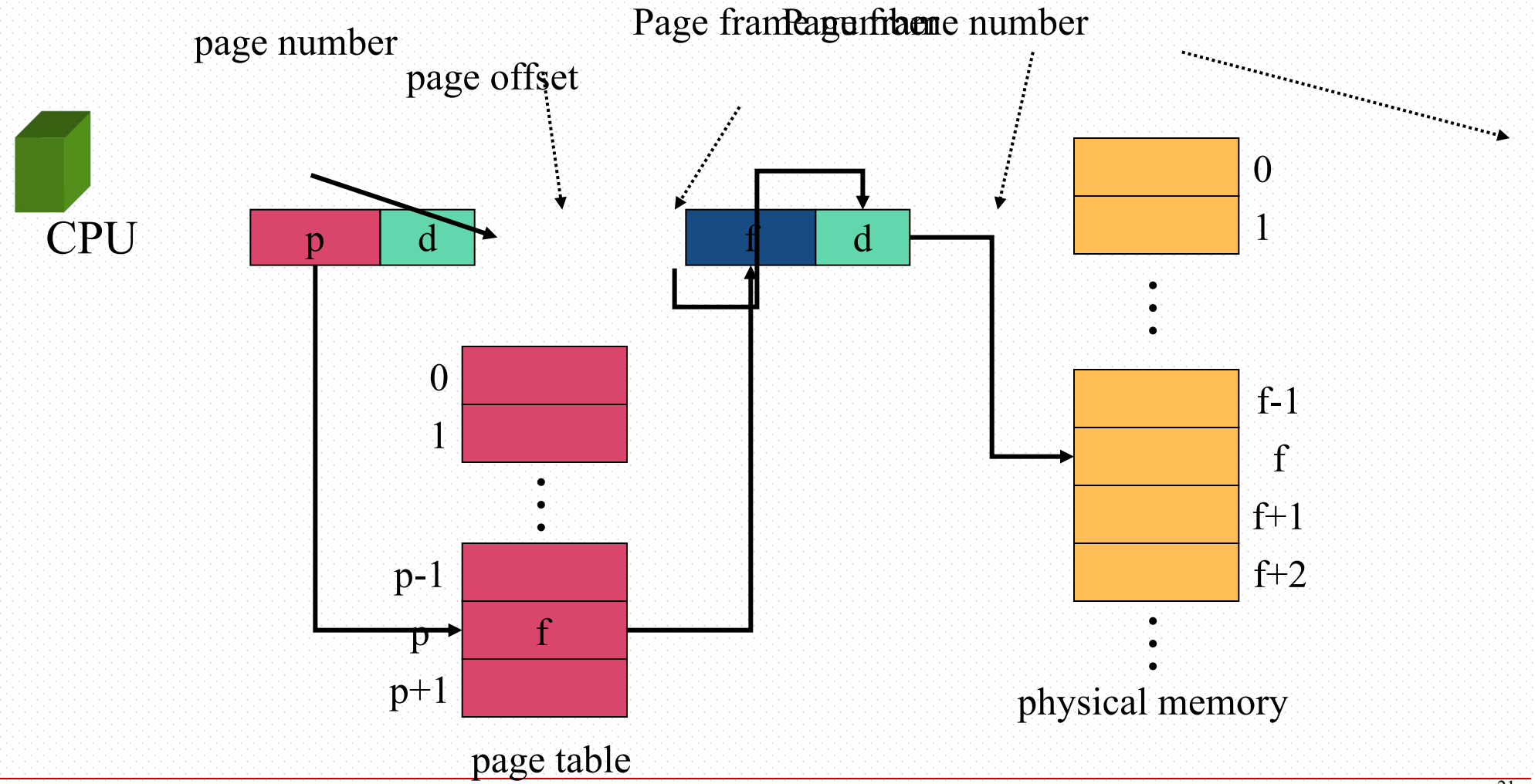
Limitations of swapping

- Problems with swapping
  - Process must fit into physical memory (impossible to run larger processes)
  - Memory becomes fragmented
    - External fragmentation: lots of small free areas
    - Compaction needed to reassemble larger free areas
  - Processes are either in memory or on disk: half and half doesn't do any good
- Overlays solved the first problem
  - Bring in pieces of the process over time (typically data)
  - Still doesn't solve the problem of fragmentation or partially resident processes

Memory & paging structures

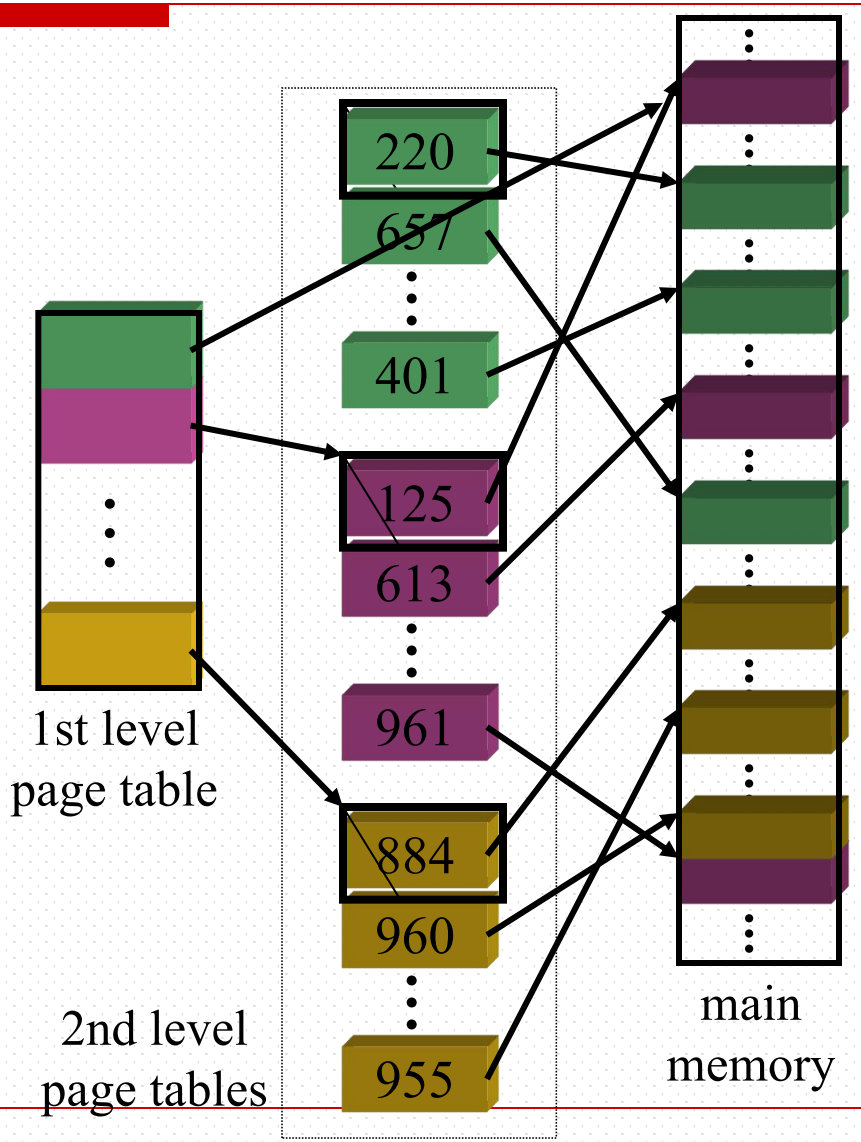


Address translation architecture



Two-level page tables

- Problem: page tables can be too large
  - 2^{32} bytes in 4KB pages need 1 million PTEs
- Solution: use multi-level page tables
  - "Page size" in first page table is large (megabytes)
  - PTE marked invalid in first page table needs no 2nd level page table
- 1st level page table has pointers to 2nd level page tables
- 2nd level page table has actual physical page numbers in it

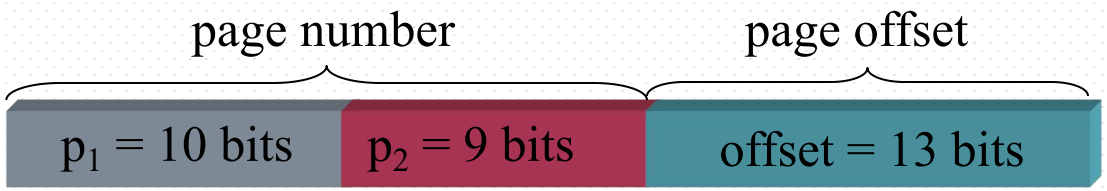


More on two-level page tables

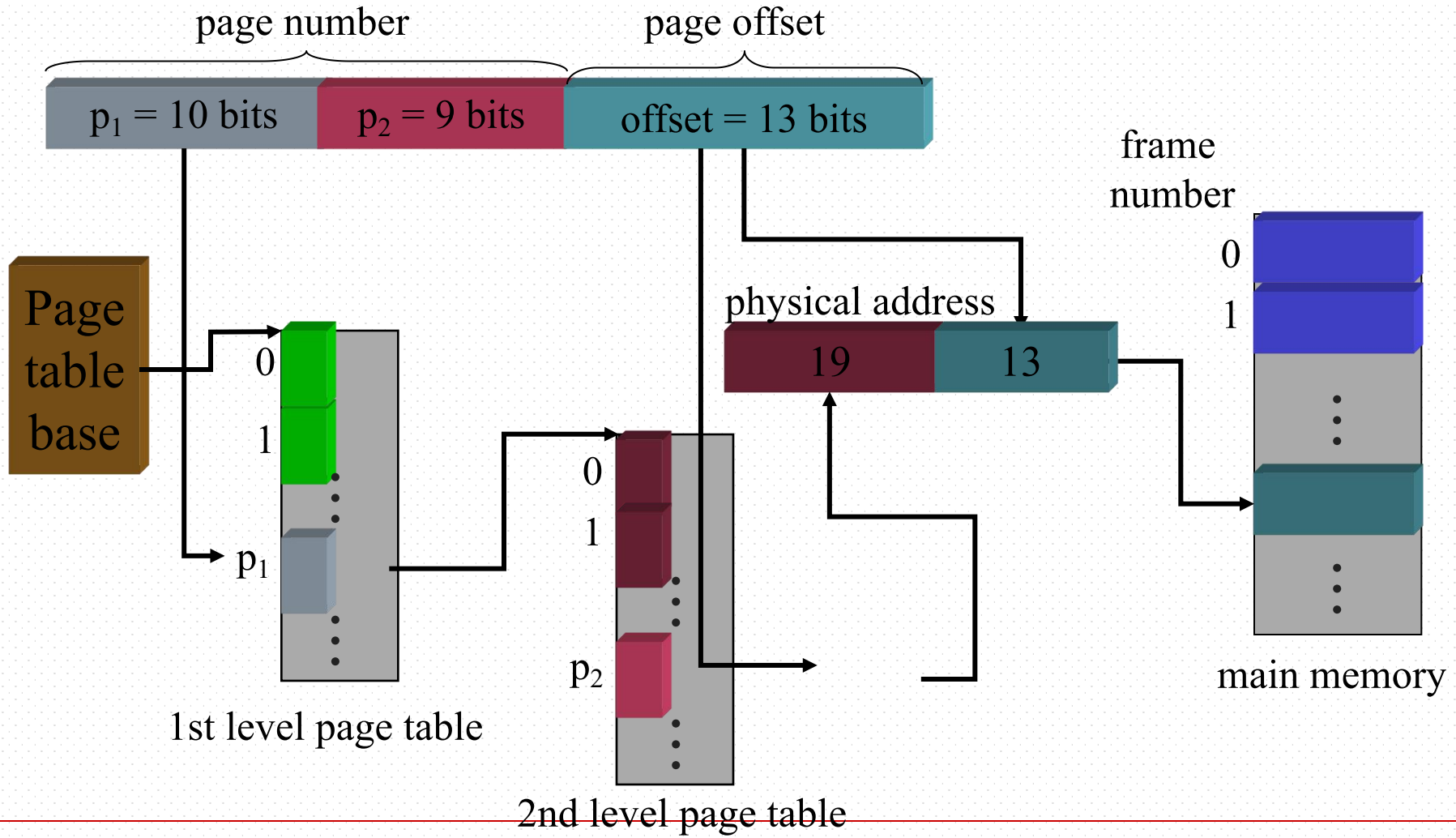
- Tradeoffs between 1st and 2nd level page table sizes
  - Total number of bits indexing 1st and 2nd level table is constant for a given page size and logical address length
  - Tradeoff between number of bits indexing 1st and number indexing 2nd level tables
    - More bits in 1st level: fine granularity at 2nd level
    - Fewer bits in 1st level: maybe less wasted space?
- All addresses in table are physical addresses
- Protection bits kept in 2nd level table

Two-level paging: example

- System characteristics
  - 8 KB pages
  - 32-bit logical address divided into 13 bit page offset, 19 bit page number
- Page number divided into:
  - 10 bit page number
  - 9 bit page offset
- Logical address looks like this:
  - p_1 is an index into the 1st level page table
  - p_2 is an index into the 2nd level page table pointed to by p_1



2-level address translation example



Implementing page tables in hardware

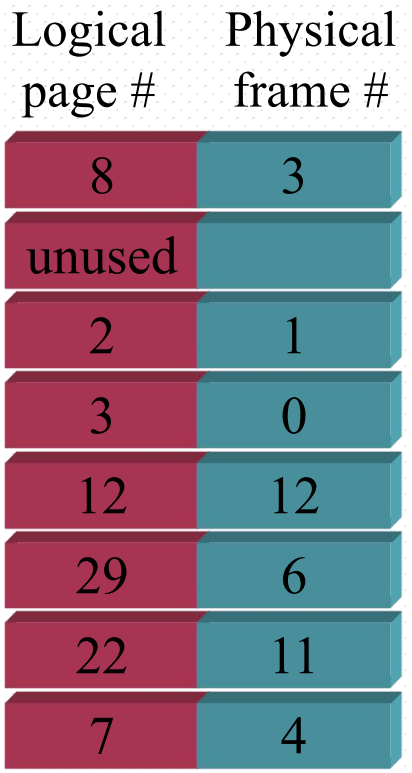
- Page table resides in main (physical) memory
- CPU uses special registers for paging
  - Page table base register (PTBR) points to the page table
  - Page table length register (PTLR) contains length of page table: restricts maximum legal logical address
- Translating an address requires two memory accesses
  - First access reads page table entry (PTE)
  - Second access reads the data / instruction from memory
- Reduce number of memory accesses
  - Can't avoid second access (we need the value from memory)
  - Eliminate first access by keeping a hardware cache (called a *translation lookaside buffer* or TLB) of recently used page table entries

Implementing page tables in hardware

- A **translation lookaside buffer** (**TLB**) is a memory cache that stores recent **translations** of virtual memory to physical addresses for faster retrieval. When a virtual memory address is referenced by a program, the search starts in the CPU. First, instruction caches are checked.

Translation Lookaside Buffer (TLB)

- Search the TLB for the desired logical page number
  - Search entries in parallel
  - Use standard cache techniques
- If desired logical page number is found, get frame number from TLB
- If desired logical page number isn't found
  - Get frame number from page table in memory
  - Replace an entry in the TLB with the logical & physical page numbers from this reference



Example TLB

Handling TLB misses

- If PTE isn't found in TLB, OS needs to do the lookup in the page table
- Lookup can be done in hardware or software
- Hardware TLB replacement
  - CPU hardware does page table lookup
  - Can be faster than software
  - Less flexible than software, and more complex hardware
- Software TLB replacement
  - OS gets TLB exception
  - Exception handler does page table lookup & places the result into the TLB
  - Program continues after return from exception
  - Larger TLB (lower miss rate) can make this feasible

Advantages of Paging

- Eliminates external fragmentation.
- Allows non-contiguous memory allocation.
- Supports virtual memory.
- Improves memory utilization.

Disadvantages of Paging

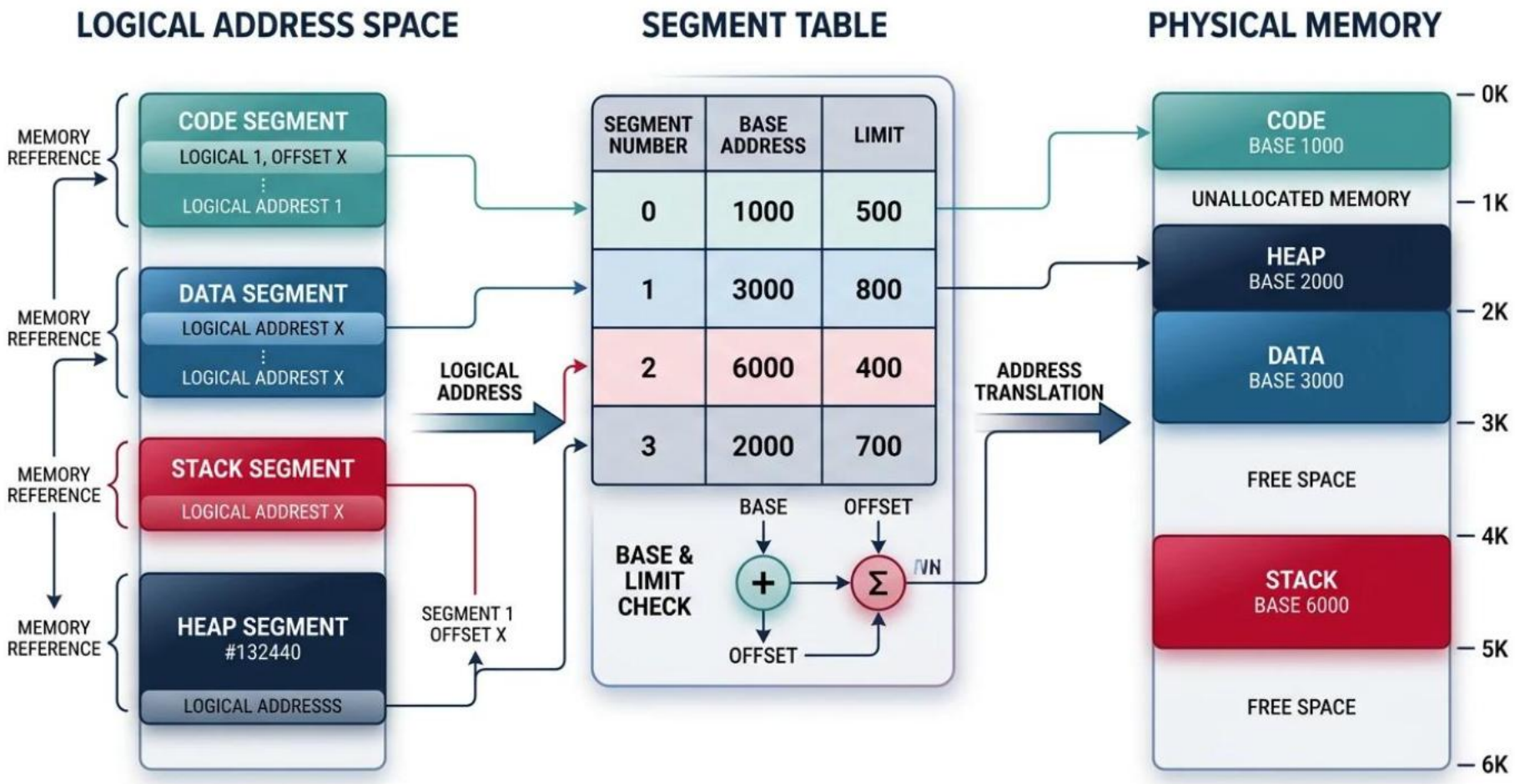
- May cause internal fragmentation.
- Requires page tables
- Address translation introduces overhead.

6. Segmentation

Segmentation is a memory management technique that divides a program into logical units called segments.

Examples of segments include:

- Code segment
- Data segment
- Stack segment
- Heap segment


LOGICAL ADDRESS SPACE
SEGMENT TABLE
PHYSICAL MEMORY
MEMORY REFERENCE
CODE SEGMENT
LOGICAL 1, OFFSET X
LOGICAL ADDREST 1
MEMORY REFERENCE
DATA SEGMENT
LOGICAL ADDREST X
LOGICAL ADDREST X
MEMORY REFERENCE
STACK SEGMENT
LOGICAL ADDREST X
MEMORY REFERENCE
HEAP SEGMENT
#132440
LOGICAL ADDRESSS
LOGICAL ADDRESS
SEGMENT 1 OFFSET X
SEGMENT NUMBER
BASE ADDRESS
LIMIT
0
1000
500
1
3000
800
2
6000
400
3
2000
700
BASE & LIMIT CHECK
BASE
OFFSET
OFFSET
ADDRESS TRANSLATION
CODE
BASE 1000
UNALLOCATED MEMORY
HEAP
BASE 2000
DATA
BASE 3000
FREE SPACE
STACK
BASE 6000
FREE SPACE
0K
1K
2K
3K
4K
5K
6K

Segmentation

-

Advantages

Supports logical program organization.

Allows sharing of program segments.

Provides protection at segment level.

Makes programs easier to manage.

Disadvantages

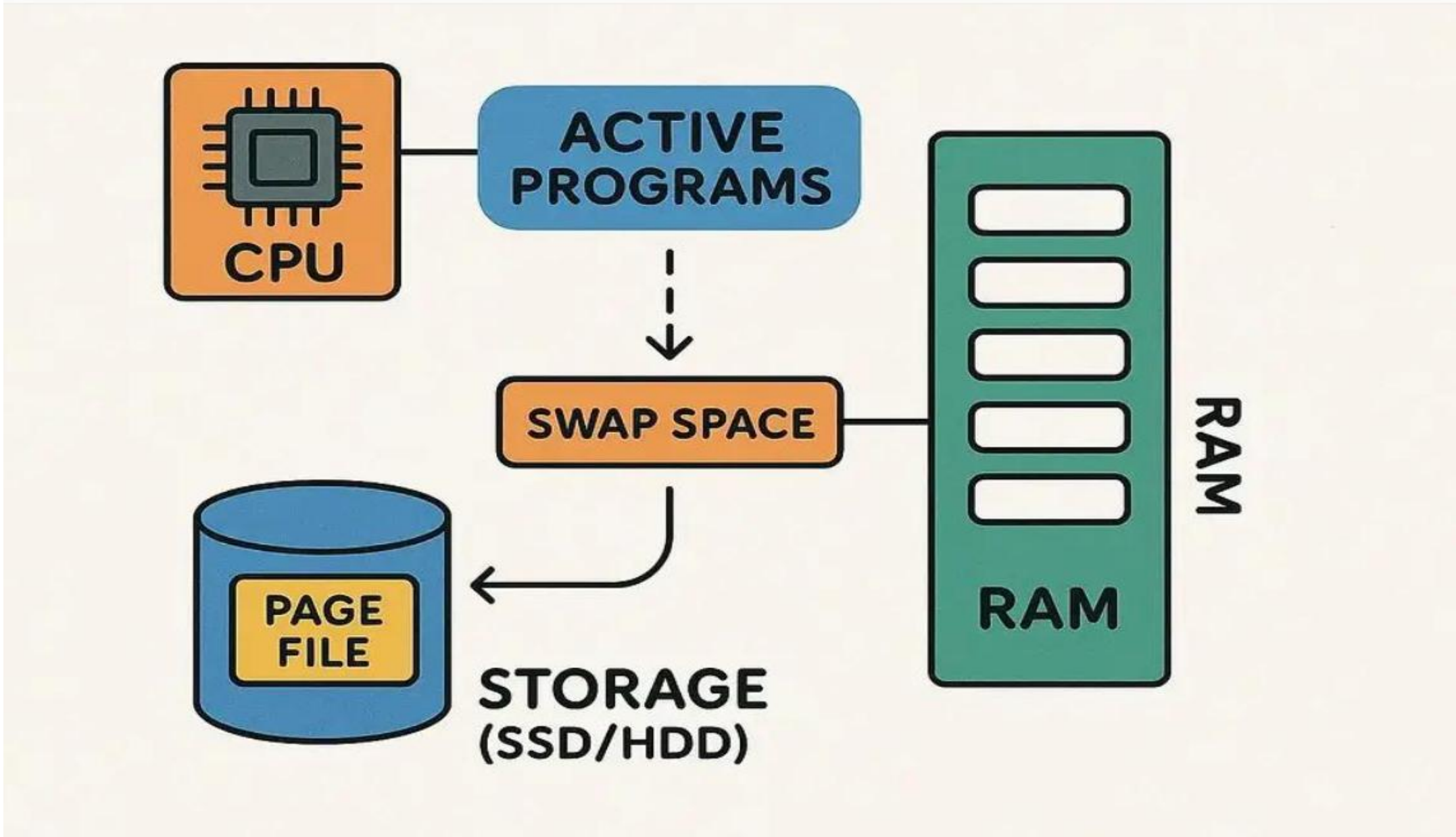
Can cause external fragmentation.

Requires segment tables.

Memory allocation can become complex.

7. Virtual Memory

- Virtual memory is a memory management technique that allows a computer to execute programs that may require more memory than the available physical RAM.
- It uses secondary storage as an extension of main memory.


CPU
ACTIVE PROGRAMS
SWAP SPACE
RAM
RAM
PAGE FILE
STORAGE
(SSD/HDD)

How Virtual Memory Works

- A program is loaded into memory.
- Only the required pages are loaded into RAM.
- Other pages remain on secondary storage
- When a program needs a page that is not in RAM, the OS loads it
- The program continues execution.

How Virtual Memory Works

Advantages

Allows large programs to execute.

Supports multitasking.

Improves RAM utilization.

Allows more processes to run simultaneously.

Disadvantages

Accessing disk storage is slower than RAM.

8. Page Replacement Algorithm

When physical memory is full and a new page must be loaded, the operating system may need to remove an existing page. This process is called page replacement.

Common page replacement algorithms include:

1. FIFO (First In, First Out)

Removes the page that entered memory first.

2. LRU (Least Recently Used)

Removes the page that has not been used for the longest period.

3. Optimal Page Replacement

Removes the page that will not be used for the longest time in the future.

4. Clock Algorithm

Uses reference bits and a circular list to select a page for replacement.

9. Page Fault

A page fault occurs when a process attempts to access a page that is not currently available in physical memory (RAM).

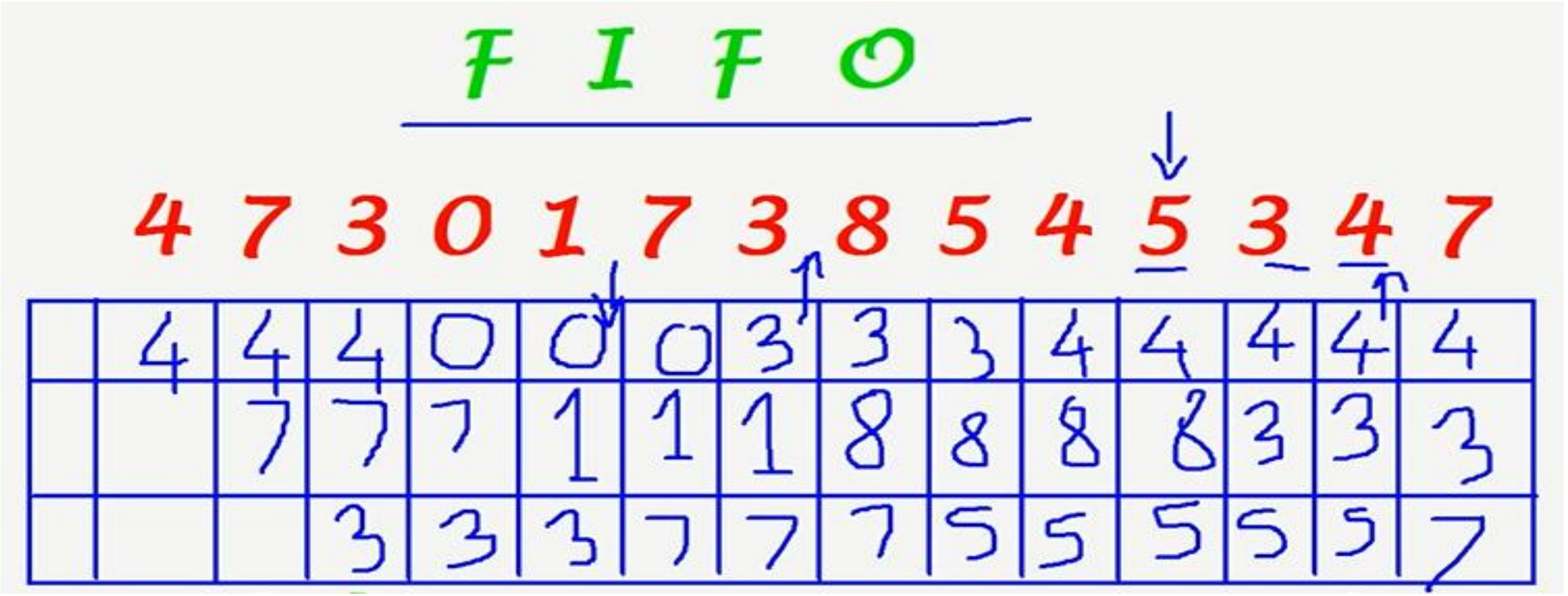
Steps in Handling a Page Fault

1. The CPU detects that the required page is missing.
2. The operating system receives a page fault interrupt.
3. The OS locates the required page on secondary storage.
4. It finds an available frame or selects a page for replacement.
5. The required page is loaded into RAM.
6. The page table is updated.
7. The interrupted process resumes execution.

Page replacement algorithms

- Page fault forces a choice
  - No room for new page (steady state)
  - Which page must be removed to make room for an incoming page?
- How is a page removed from physical memory?
  - If the page is unmodified, simply overwrite it: a copy already exists on disk
  - If the page has been modified, it must be written back to disk: prefer unmodified pages?
- Better not to choose an often used page
  - It'll probably need to be brought back in soon

First-In, First-Out (FIFO) Example



Optimal page replacement algorithm

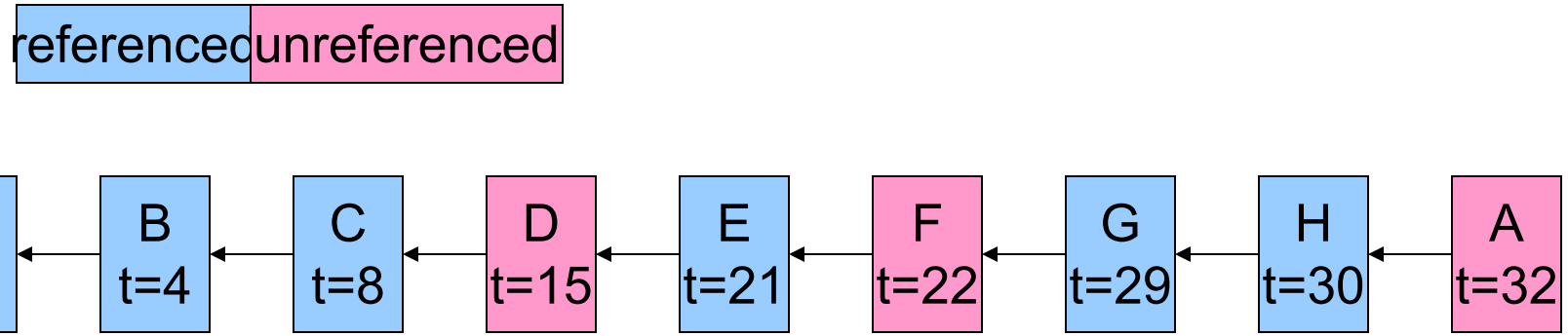
- The optimal algorithm for page replacement requires knowledge of the entire page reference stream in advance.
- When a page fault occurs and there are no empty frames for the process, **the algorithm looks ahead in the page reference stream to find out about future references to the pages currently in physical memory.**
- The algorithm uses this "future" knowledge to choose the optimal page to be removed from memory.
- The approach used is to replace the page in memory that will not be used for the longest period.
- The optimal algorithm is not normally realizable in practice, but it serves as a reference for comparison with the other algorithms.

Not-recently-used (NRU) algorithm

- Each page has reference bit and dirty bit
  - Bits are set when page is referenced and/or modified
- Pages are classified into four classes
  - 0: not referenced, not dirty
  - 1: not referenced, dirty
  - 2: referenced, not dirty
  - 3: referenced, dirty
- Clear reference bit for all pages periodically
  - Can’t clear dirty bit: needed to indicate which pages need to be flushed to disk
  - Class 1 contains dirty pages where reference bit has been cleared
- Algorithm: remove a page from the lowest non-empty class
  - Select a page at random from that class
- Easy to understand and implement
- Performance adequate (though not optimal)

Second chance page replacement

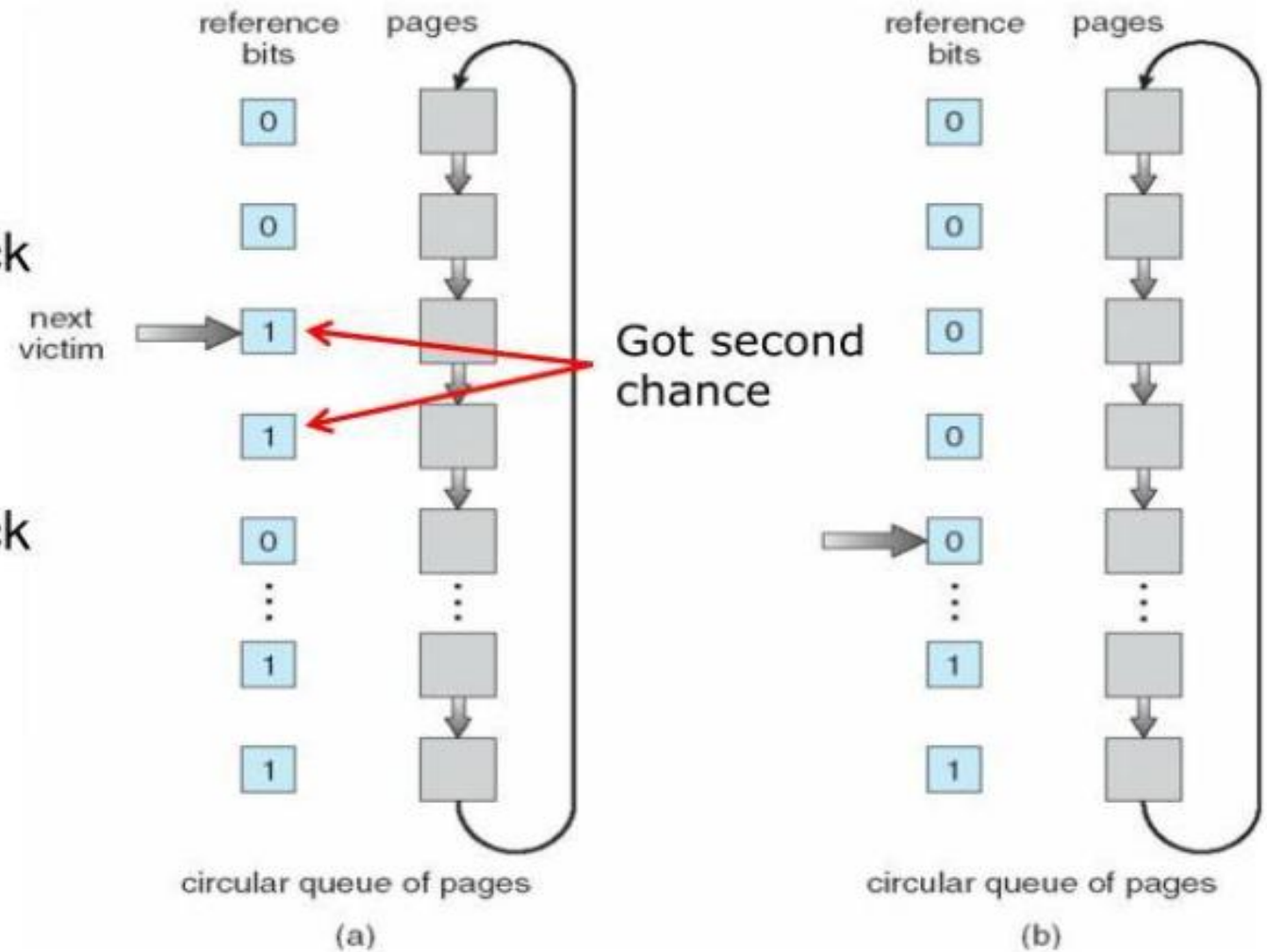
- Modify FIFO to avoid throwing out heavily used pages
  - If reference bit is 0, throw the page out
  - If reference bit is 1
    - Reset the reference bit to 0
    - Move page to the tail of the list
    - Continue search for a free page
- Still easy to implement, and better than plain FIFO



Second-Chance (clock) Page-Replacement Algorithm

Second chance

- Need reference bit
- Clock replacement
- If page to be replaced (in clock order) has reference bit = 0 then:
  - Replace the page
- If page to be replaced (in clock order) has reference bit = 1 then:
  - set reference bit 0
  - leave page in memory
  - It arrival time is reset to current the current time

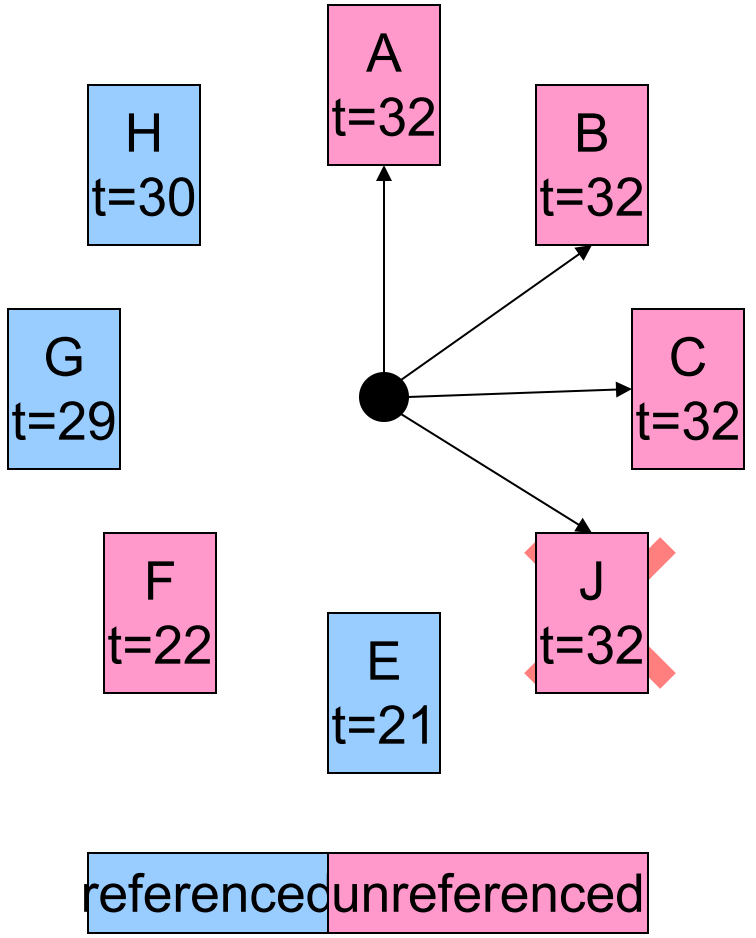


First-In, First-Out (FIFO) algorithm

- The FIFO algorithm is the simplest for replacement. When a page fault occurs and there are no empty frames for the process, the page selected to be replaced is the one that has been in memory the longest time-the oldest page.
- This selection of the page to replace is completely independent of the locality of the process.
- Thus, it does not follow very well the general behavior of processes. In practice, FIFO has the worst performance compared to the other algorithms.
- The only advantage of this algorithm is its simplicity and ease of implementation.

Clock algorithm

- Same functionality as second chance
- Simpler implementation
  - “Clock” hand points to next page to replace
  - If R=0, replace page
  - If R=1, set R=0 and advance the clock hand
- Continue until page with R=0 is found
  - This may involve going all the way around the clock…



Least Recently Used (LRU)

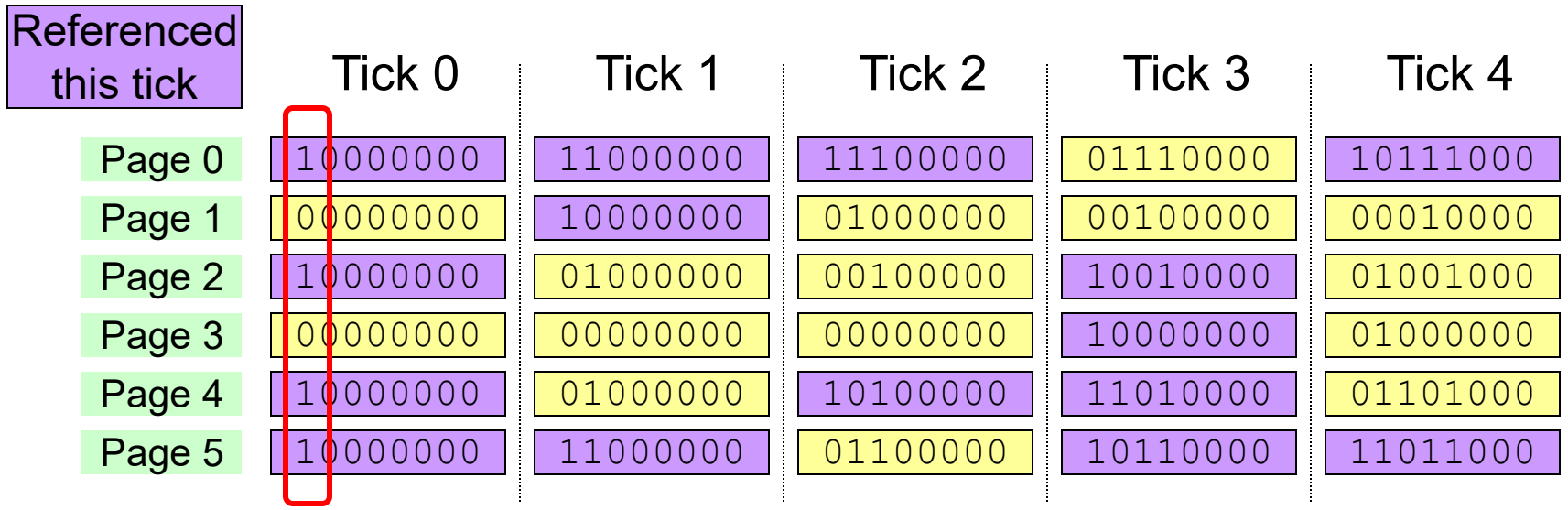
- Assume pages used recently will used again soon
  - Throw out page that has been unused for longest time
- Must keep a linked list of pages
  - Most recently used at front, least at rear
  - Update this list every memory reference!
    - This can be somewhat slow: hardware has to update a linked list on every reference!
- Alternatively, keep counter in each page table entry
  - Global counter increments with each CPU cycle
  - Copy global counter to PTE counter on a reference to the page
  - For replacement, evict page with lowest counter value

Simulating LRU in software

- Few computers have the necessary hardware to implement full LRU
  - Linked-list method impractical in hardware
  - Counter-based method could be done, but it's slow to find the desired page
- Approximate LRU with Not Frequently Used (NFU) algorithm
  - At each clock interrupt, scan through page table
  - If R=1 for a page, add one to its counter value
  - On replacement, pick the page with the lowest counter value
- Problem: no notion of age—pages with high counter values will tend to keep them!

Aging replacement algorithm

- Reduce counter values over time
  - Divide by two every clock cycle (use right shift)
  - More weight given to more recent references!
- Select page to be evicted by finding the lowest counter value
- Algorithm is:
  - Every clock tick, shift all counters right by 1 bit
  - On reference, set leftmost bit of a counter (can be done by copying the reference bit to the counter at the clock tick)



10. Memory Fragmentation

Memory fragmentation occurs when available memory is divided into small pieces, making efficient allocation difficult.

- There are two main types:

a) Internal Fragmentation

Occurs when a process is allocated more memory than it actually needs, leaving unused space inside the allocated block.

b) External Fragmentation

Occurs when free memory is divided into small, non-contiguous blocks, making it difficult to allocate a large continuous memory block.

11 Memory Protection

Memory protection ensures that one process cannot access or modify the memory of another process without authorization.

Benefits:

- Prevents unauthorized access.
- Improves system security.
- Prevents accidental data modification.
- Increases system stability.

12.Memory Sharing

Memory sharing allows multiple processes to access a common memory region.

Examples:

- Shared libraries.
- Inter-process communication (IPC).
- Shared memory segments.

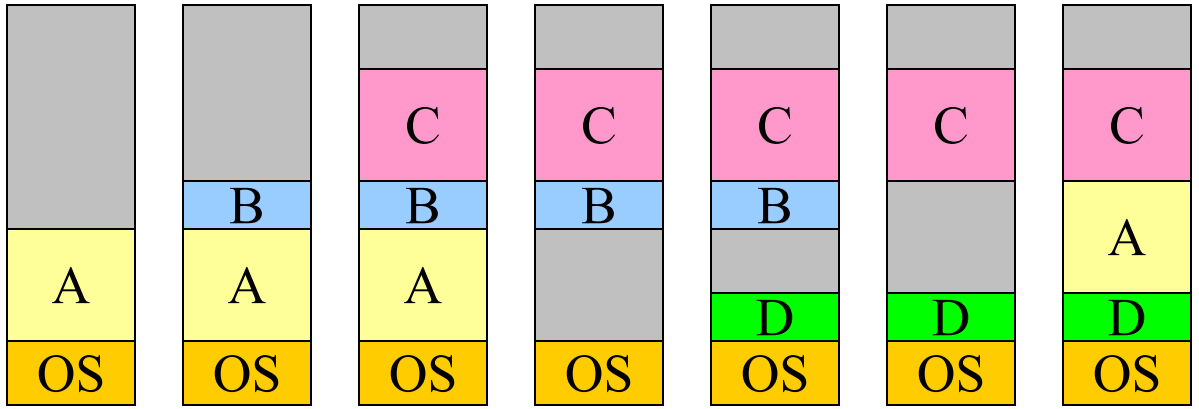
13.Swapping

Swapping is a memory management technique in which a process is temporarily moved from RAM to secondary storage and later brought back into RAM.

Swapping Process

1. The operating system selects a process.
2. The process is moved from RAM to secondary storage.
3. Memory space becomes available.
4. Another process is loaded into RAM.
5. The suspended process can later be brought back.

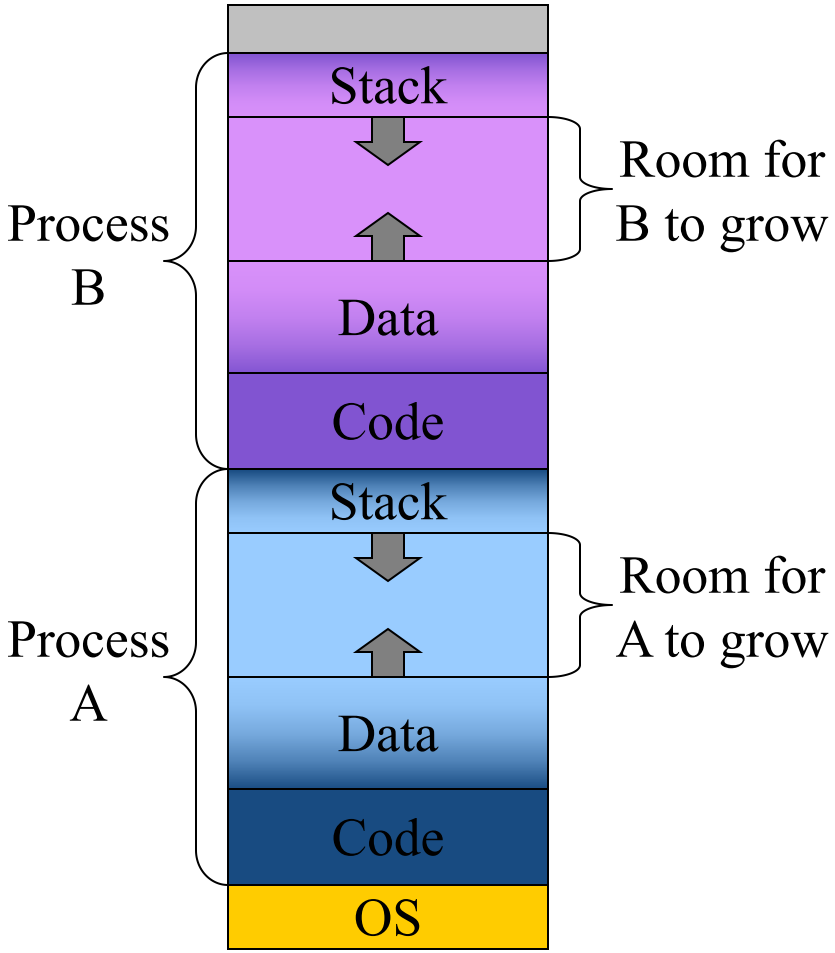
Swapping



- Memory allocation changes as
  - Processes come into memory
  - Processes leave memory
    - Swapped to disk
    - Complete execution
- Gray regions are unused memory

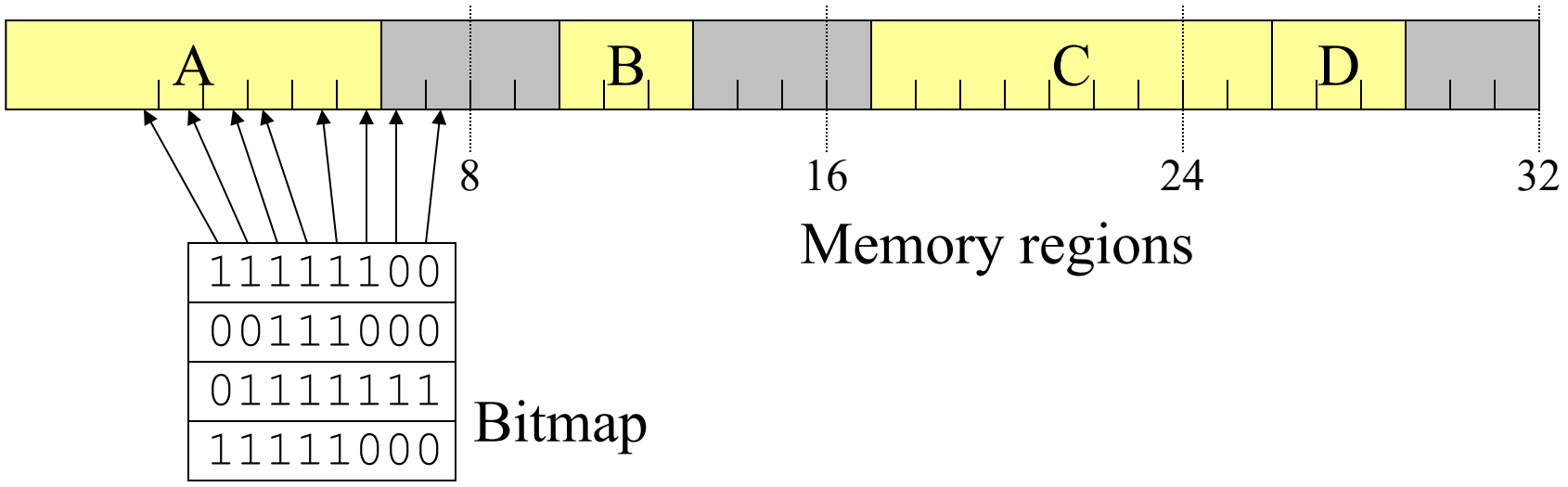
Swapping: leaving room to grow

- Need to allow for programs to grow
  - Allocate more memory for data
  - Larger stack
- Handled by allocating more space than is necessary at the start
  - Inefficient: wastes memory that's not currently in use
  - What if the process requests too much memory?



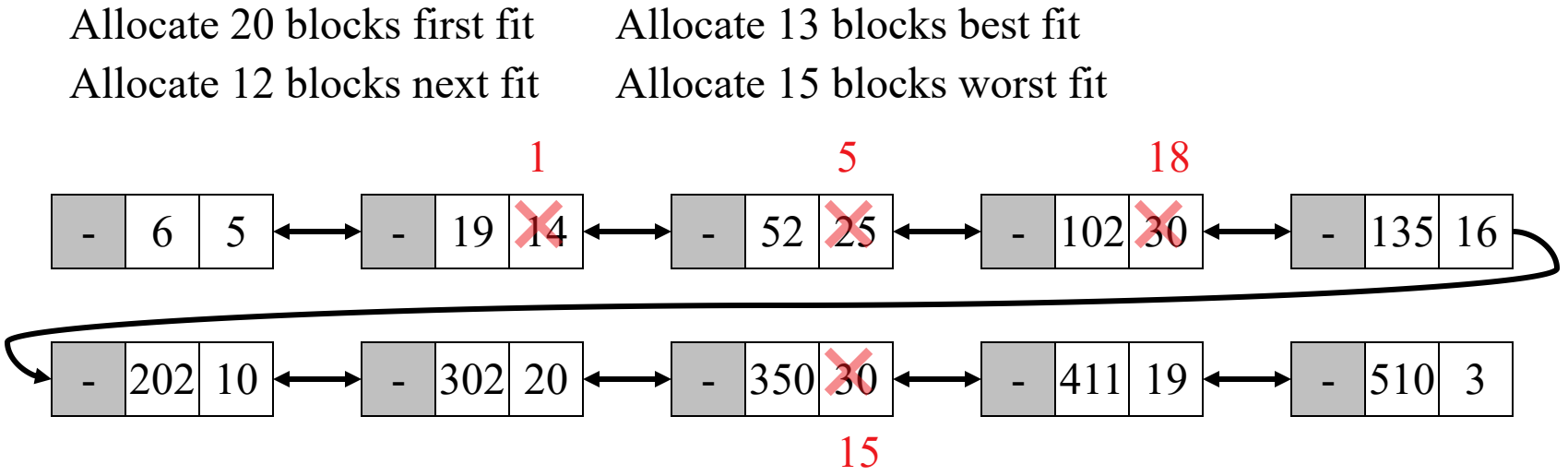
Tracking memory usage: bitmaps

- Keep track of free / allocated memory regions with a bitmap
  - One bit in map corresponds to a fixed-size region of memory
  - Bitmap is a constant size for a given amount of memory regardless of how much is allocated at a particular time
- Chunk size determines efficiency
  - At 1 bit per 4KB chunk, we need just 256 bits (32 bytes) per MB of memory
  - For smaller chunks, we need more memory for the bitmap
  - Can be difficult to find large contiguous free areas in bitmap



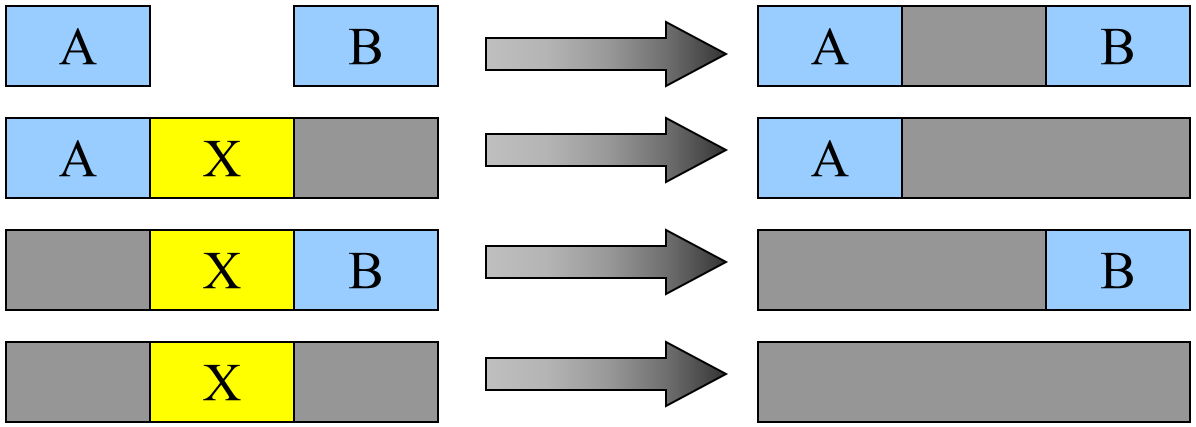
Allocating memory

- Search through region list to find a large enough space
- Suppose there are several choices: which one to use?
  - **First fit**: the first suitable hole on the list
  - **Next fit:** the first suitable after the previously allocated hole
  - **Best fit:** the smallest hole that is larger than the desired region (wastes least space?)
  - **Worst fi**t: the largest available hole (leaves largest fragment)
- Option: maintain separate queues for different-size holes



Freeing memory

- Allocation structures must be updated when memory is freed
- Easy with bitmaps: just set the appropriate bits in the bitmap
- Linked lists: modify adjacent elements as needed
  - Merge adjacent free regions into a single region
  - May involve merging two regions with the just-freed area

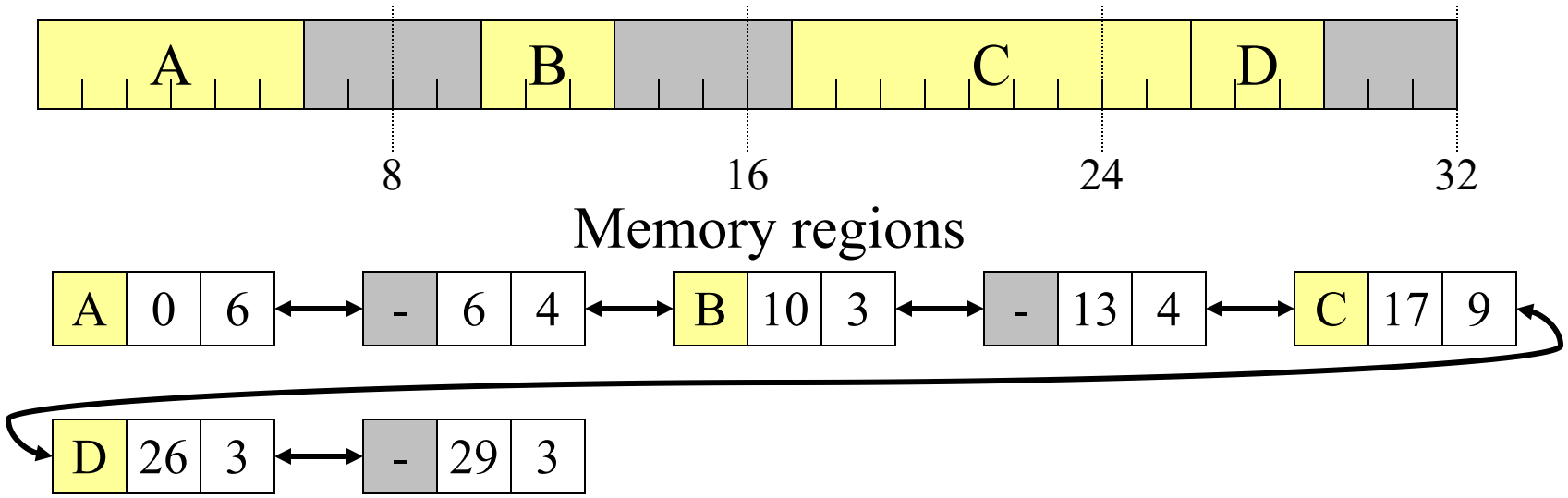


Virtual memory

- **Basic idea**: allow the OS to hand out more memory than exists on the system
- Keep recently used stuff in physical memory
- Move less recently used stuff to disk
- Keep all of this hidden from processes
  - Processes still see an address space from 0 – max address
  - Movement of information to and from disk handled by the OS without process help
- Virtual memory (VM) especially helpful in multiprogrammed system
  - CPU schedules process B while process A waits for its memory to be retrieved from disk

Tracking memory usage: linked lists

- Keep track of free / allocated memory regions with a linked list
  - Each entry in the list corresponds to a contiguous region of memory
  - Entry can indicate either allocated or free (and, optionally, owning process)
  - May have separate lists for free and allocated areas
- Efficient if chunks are large
  - Fixed-size representation for each region
  - More regions => more space needed for free lists



14. Thrashing

Thrashing occurs when the operating system spends more time moving pages between RAM and secondary storage than executing actual processes.

Causes of Thrashing

- Insufficient physical memory.
- Too many active processes.
- Excessive page faults.
- Poor memory allocation.

Solutions

- Increase physical RAM.
- Reduce the number of active processes.
- Improve page replacement algorithms.
- Allocate sufficient frames to processes.

15. Summary

- Memory management is a fundamental responsibility of an operating system. It ensures that computer memory is allocated, utilized, protected, and released efficiently.
- The major concepts include memory allocation, paging, segmentation, virtual memory, page replacement, fragmentation, swapping, and memory protection.
- Understanding these concepts helps understand how operating systems manage multiple programs and improve computer performance.

END